
Workgroup:	Fast Network Notifications
Internet-Draft:	draft-song-fann-framework-00
Published:	12 June 2026
Intended Status:	Informational
Expires:	14 December 2026
Author:	H. Song <i>Futurewei Technologies</i>

A Framework for Fast Network Notifications

Abstract

Many network applications, ranging from Artificial Intelligence (AI) / Machine Learning (ML) training and inference to large-scale cloud services, require high bandwidth, low delay, low jitter, and minimal packet loss. Meeting these requirements depends on the network's ability to adapt rapidly to faults, signal degradation, and congestion. The companion problem statement describes why existing mechanisms are too slow, too coarse, or too resource-intensive to react within the timescales at which modern forwarding hardware can detect and disseminate intended conditions.

This document defines a framework for Fast Network Notifications (FANN). It describes a reference architecture, the functional roles involved in generating and consuming notifications, an information model, delivery and scoping models, procedures for discovery, registration, and subscription, and the integration of fast network notifications with existing Layer 2 to 4 mechanisms. This framework is intended to guide the development of one or more fast network notification protocol specifications.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 14 December 2026.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
1.1. Scope	4
1.2. Requirements Language	4
2. Terminology	4
3. Fast Network Notification Framework	5
3.1. Design Principles and Goals	5
3.2. Deployment Scenarios	6
3.3. Reference Architecture	7
3.3.1. Functional Roles	7
3.3.2. Notification Lifecycle	8
3.3.3. Detection Assumptions and Constraints	8
3.4. Information Model	9
3.5. Delivery and Scoping	9
3.5.1. Delivery Modes	9
3.5.2. Transport Considerations	10
3.5.3. Notification Domain and Isolation	11
3.6. Discovery, Registration, and Subscription	11
3.7. Loop Prevention, Filtering, and Damping	11
4. Realization and Operational Considerations	12
4.1. Integration with Existing Technologies	12

4.2. Candidate Realization Approaches	13
4.3. Illustrative Applications	14
4.4. Scaling Considerations	15
4.5. Operational Considerations	16
5. Security Considerations	16
6. IANA Considerations	17
7. References	17
7.1. Normative References	17
7.2. Informative References	17
Acknowledgements	19
Author's Address	19

1. Introduction

Modern high-performance networks, in particular data center (DC) and data center interconnect (DCI) fabrics serving AI/ML and cloud workloads, demand rapid adaptation to changing network conditions. A single fiber link failure, signal degradation, or transient congestion event can stall a distributed training job, waste compute and energy, and degrade service experience [I-D.ietf-rtgwg-net-notif-ps].

Contemporary forwarding hardware can detect link failures, signal degradation reported as link errors, queue buildup, microbursts, and output-queue congestion at microsecond to sub-millisecond timescales. However, the time required to disseminate this information to the remote nodes that can act on it typically far exceeds the detection time. This gap between detection and reaction is the central problem that fast network notifications address.

The Fast Network Notifications Problem Statement [I-D.ietf-rtgwg-net-notif-ps] documents the need for a fast notification mechanism and the limitations of existing approaches. The companion requirements [I-D.geng-fantel-fantel-requirements] and gap analysis [I-D.geng-fantel-fantel-gap-analysis] documents elaborate the requirements and the deficiencies of current technologies. Built on these documents, this document defines a framework to describe the overall architecture, the functional roles, the information carried, how notifications are delivered and scoped, and how the mechanism integrates with existing protocols and technologies across layers.

This informational document does not define a wire protocol, encoding, or YANG model. Those are expected to be specified in separate protocol and management documents that build on this framework.

1.1. Scope

This framework applies to limited-domain networks under a single administrative control, consistent with the deployment assumptions of the FANN charter. It prioritizes the requirements of DC and DCI networks where rapid responsiveness is critical, while remaining applicable to other deployments such as wide-area backbone networks.

The framework initially targets notifications for link failures, signal degradation reported as link errors, and port queue congestion, while remaining extensible to additional conditions in the future. The specific actions a recipient takes in response to a notification (for example fast reroute, adaptive load balancing, or rate adjustment) are out of scope of this framework; they are the responsibility of the consuming subsystem and the protocols that realize those actions.

In this document, "fast" does not denote a single rigid numerical threshold. It characterizes a class of mechanisms designed to minimize notification delivery time so that the latency is on the order of microseconds to milliseconds, depending on the operational objective and the diameter of the notification domain, and is substantially shorter than the Round-Trip Time (RTT) of the affected traffic.

This framework is solution-agnostic. It defines the functional roles, information model, and delivery and scoping models that a fast network notification solution is expected to instantiate, but it does not specify, mandate, or endorse any particular protocol, encoding, or solution document. It is intentionally general so that a range of realization approaches, for example control-plane, forwarding-plane, in-band, or telemetry-assisted mechanisms, can conform to it, potentially in combination, without conflicting with one another or with this framework. Specific solutions are developed in separate documents; such documents are expected to map their behavior onto the roles and models defined here, and any capability they require that is not yet covered is expected to be accommodated as an extension of this framework rather than a departure from it.

1.2. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

2. Terminology

This document uses the following terms.

Fast Network Notification (FANN): An event-driven message that conveys a locally detected adverse network condition, or the recovery from such a condition, to one or more remote nodes within a defined notification domain, with the objective of low-latency delivery suitable for action in the forwarding plane.

Notification Originator: A functional role that detects an adverse condition (or its recovery) and generates a fast network notification. Also referred to as the originating or reporting node.

Notification Consumer: A functional role that receives a fast network notification and may take action based on it. Also referred to as the recipient node.

Notification Relay: A functional role that forwards or redistributes a notification toward additional consumers, optionally applying filtering, damping, or aggregation. A node may be both a relay and a consumer.

Notification Domain: A bounded region of the network, under a single administrative control, within which fast network notifications are generated, distributed, and consumed, and outside of which they are not propagated without explicit policy.

Notification Controller: An optional control-plane or management-plane entity that assists with discovery, registration, subscription, policy, and global optimization. It is not required to be on the fast notification delivery path.

Event: A change in network condition detected by a node, such as a link failure, signal degradation, or output-queue congestion, including the recovery from a previously reported condition.

The terms BFD [[RFC5880](#)], ECN [[RFC3168](#)], FRR [[RFC4090](#)] [[RFC5714](#)], and IOAM [[RFC9197](#)] are used as defined in their respective references.

3. Fast Network Notification Framework

This chapter defines the core of the framework: its design principles, the deployment scenarios it serves, the functional reference architecture, the information carried in notifications, and the models for delivering, scoping, discovering, and controlling them.

3.1. Design Principles and Goals

The framework is guided by the following principles, derived from the problem statement [[I-D.ietf-rtgwg-net-notif-ps](#)] and requirements [[I-D.geng-fantel-fantel-requirements](#)].

Event-driven, not periodic: Notifications are generated in response to detected events. This distinguishes fast network notifications from preconfigured periodic mechanisms such as BFD [[RFC5880](#)], which detect rather than disseminate.

Forwarding-plane optimized: The primary design point is fast generation and consumption in the forwarding plane, ideally in hardware, to meet responsiveness targets. Consumption by the control plane and management plane is a secondary objective and **MUST NOT** compromise routing stability.

Lightweight and bounded: Notification messages are compact and the system is designed to bound the load it places on the network, especially during the very events it reports. The notification system **MUST NOT** exacerbate a failure or congestion event.

Action-agnostic: The framework conveys information; it does not mandate a specific reaction. A notification **MAY** explicitly indicate a recommended action, or the action **MAY** be determined implicitly by the consumer from the information carried.

Extensible: The information model and event taxonomy are extensible to additional conditions, metrics, and scopes without redefining the architecture.

Complementary: Fast network notifications complement, and do not replace, existing OAM, control-plane, and telemetry mechanisms. They bridge the time gap between event onset and slower control-plane or telemetry-driven responses.

Scoped and isolated: Notifications are confined to a notification domain. Domain identification and isolation are first-class concerns.

Decoupled from routing convergence: Fast-changing network state **SHOULD** be conveyed by mechanisms that are separate from the routing protocol database and their own flooding and best-path computation, so that high-frequency or transient events do not introduce churn, instability, or excessive recomputation in the routing control plane. This separation lets notifications be generated and refreshed at a fast pace independently of routing convergence, while any consumption by routing remains a secondary objective bounded by the routing-stability constraint above.

3.2. Deployment Scenarios

Fast network notifications apply across a range of network scenarios, but the time budget, processing constraints, and the mechanisms that are practical differ substantially between them. This framework does not assume one-size-fits-all: the scenarios below have materially different characteristics, and a given deployment is expected to select the delivery mode, scope, and realization mechanism appropriate to its scenario rather than apply a single mechanism everywhere.

Intra-data-center fabric: Within a single DC fabric (for example a Clos topology), originators and consumers are typically a small number of hops apart, propagation delay is very low, and forwarding hardware can both detect and consume notifications. This scenario has the tightest time budget (sub-millisecond) and is the most amenable to forwarding-plane, in-band, or scoped-flooding delivery with action such as adaptive load balancing or local repair. The dominant challenge is volume and rate of change at scale rather than propagation distance.

Single-hop DCI / point-to-point WAN: Between two sites or routers connected by one (logical) hop, the recipient set is small and often known in advance, favoring unicast or a directed notification to the upstream or ingress node. The time budget is dominated by the link propagation delay, which is fixed; the design goal is to add minimal processing delay on top of it so the notification still beats the affected traffic's E2E reaction loop.

Multi-hop / arbitrary WAN paths: When notifications must reach nodes several hops away or across a wider domain, propagation delay, the number of potential recipients, and the risk of notification storms all grow. Time budgets are typically milliseconds rather than sub-millisecond, and subscription, relaying with filtering/aggregation, and bounded scoping

become essential. Some timeliness targets achievable intra-DC may simply not be feasible here; the framework expects such cases to use different, more conservative techniques, and the feasibility of meeting a specific target is itself scenario-dependent.

The functional roles (Section 3.3), information model (Section 3.4), and delivery modes (Section 3.5) defined in this document are common across scenarios, but their realization and the achievable latency are not. Where a requirement (for example a sub-millisecond target) is stated, it **SHOULD** be understood as scoped to the scenario in which it is feasible.

3.3. Reference Architecture

3.3.1. Functional Roles

The framework defines four functional roles. A single physical or virtual network element **MAY** implement more than one role.

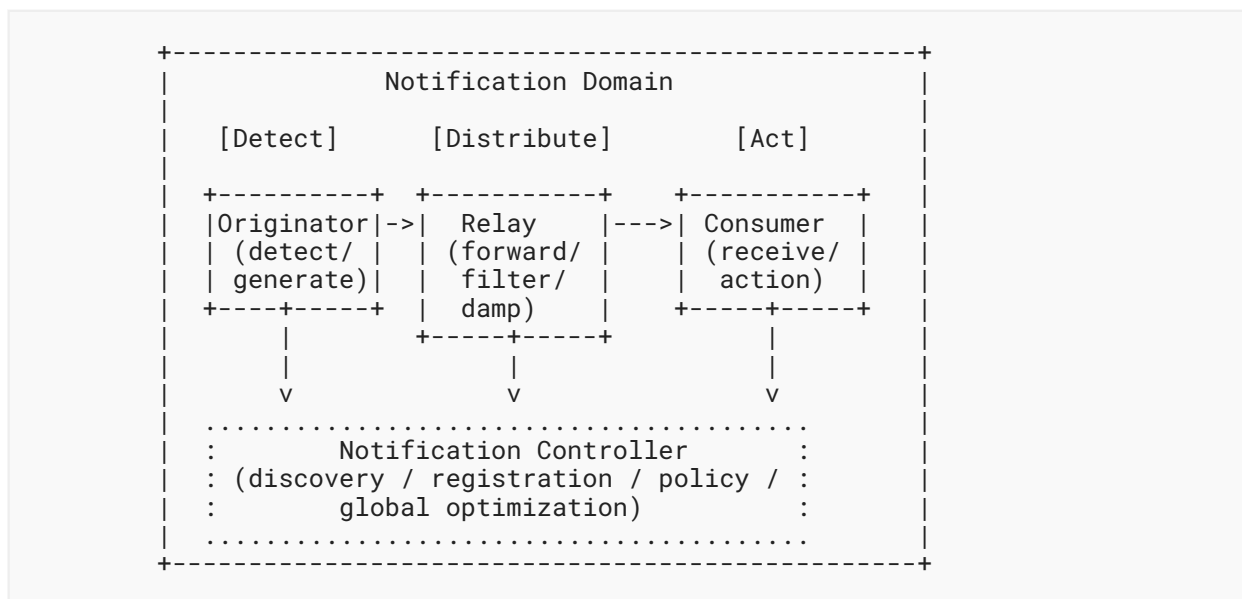


Figure 1: FANN functional roles within a domain

Notification Originator: Detects an event using local detection mechanisms (for example link fault detection, error counters, queue occupancy thresholds, or BFD [RFC5880] as a detection input) and generates a notification. The originator determines, by policy or signaling, the set of consumers and the delivery mode, and applies origination-side controls such as damping and rate limiting.

Notification Relay: Receives a notification and forwards it toward additional consumers. A relay **MAY** filter, aggregate, deduplicate, or damp notifications. Relays enable hop-by-hop and scoped-flooding delivery and allow load to be bounded inside the domain.

Notification Consumer: Receives a notification and may act on it in the data plane (for example local repair, ECMP rebalancing, flow steering), and/or pass the information to the control plane or management plane. A consumer **MAY** also be a relay.

Notification Controller: An optional entity that supports discovery, registration, subscription, and policy distribution, and that may consume notifications for global traffic-engineering or load-balancing optimization. The controller is not required to be on the fast delivery path and **SHOULD NOT** be a single point of failure for forwarding-plane reactions.

3.3.2. Notification Lifecycle

A fast network notification proceeds through the following stages.

1. **Detection.** A node observes an event at the forwarding plane using a local detection mechanism. Detection mechanisms are out of scope of this framework, but their output is the trigger for notification generation.
2. **Generation.** The originator constructs a notification populated from the information model ([Section 3.4](#)), subject to origination policy, damping, and rate limiting.
3. **Delivery.** The notification is delivered to the intended consumers using one of the delivery modes ([Section 3.5](#)), possibly via one or more relays.
4. **Consumption.** A consumer parses the notification and decides whether and how to act, based on the information carried and on any local state it holds.
5. **Action.** The consumer performs an action (out of scope of this framework) and/or relays the notification further.
6. **Recovery and withdrawal.** When the condition clears, the originator generates a recovery notification so consumers can revert or update their state. Recovery notifications are subject to damping so that flapping conditions do not generate excessive traffic.

3.3.3. Detection Assumptions and Constraints

The mechanisms by which a node detects an event are out of scope of this framework, but the framework assumes their existence and depends on their characteristics. Two assumptions are important.

First, the E2E responsiveness of a fast notification system is bounded by detection time as well as delivery time: a notification cannot be faster than the moment the originating node becomes aware of the condition. Detection latency, accuracy, and false-positive behavior therefore directly shape what the notification system can achieve, and an event that is detected slowly or unreliably limits the value of fast delivery.

Second, detection itself has a cost that interacts with scaling. For example, achieving fast liveness detection by running BFD [[RFC5880](#)] at very short transmit intervals consumes forwarding and control resources and does not by itself notify any node beyond the BFD endpoints. Driving detection intervals down to obtain faster notification can impose significant load, and this trade-off between detection speed and detection cost **SHOULD** be considered together with the

notification load discussed in [Section 4.4](#). Where hardware can be event-driven and detect a condition (for example loss of signal, FEC errors, or queue-occupancy thresholds) directly, it is preferable to polling-based or session-based detection tuned to an aggressive interval.

3.4. Information Model

A fast network notification carries one or more information elements. For a given scenario some elements are mandatory and others optional; the framework does not require all elements in every notification. The detailed encoding is left to protocol specifications. The information elements are:

Event Type: The class of event, for example failure, signal degradation, congestion, or performance degradation, and whether the notification reports onset or recovery. The event taxonomy is extensible.

Location of Event: An identifier of where the event occurred, for example a link, node, interface, or queue identifier. Location identifiers **SHOULD** be interpretable by consumers within the notification domain.

Fine-grained Network Status: Quantifiable metrics such as link utilization, available bandwidth, link capacity, queue length, level of congestion, link or node delay, jitter, and packet loss. Conveying such quantitative metrics, rather than a binary up/down indication, enables graduated and proportional responses such as weighted load-sharing adjustments.

Path Identification: Identification of the path affected by the event, allowing consumers to scope their reaction to specific paths.

Flow/Service Identification: Identification of an affected flow (for example a 5-tuple) or service, allowing differentiated, per-flow or per-service responses.

Timing and Validity: Optional event timestamp and a validity or hold time after which the reported condition should be considered stale absent refresh or recovery.

Action Hint: An optional explicit indication of a recommended action. When absent, the consumer determines the action implicitly from the other elements.

Origin and Sequence: Originator identity and a sequence or epoch indicator to support ordering, deduplication, and loop detection at relays and consumers.

A consistent information model across implementations is necessary for interoperability; defining the normative model and encodings is a task for the protocol specification.

3.5. Delivery and Scoping

3.5.1. Delivery Modes

Depending on the position and number of consumers, the framework supports the following delivery modes. A scenario **MAY** use more than one.

Unicast: Direct delivery to a single consumer. Suitable when the originator knows the specific node that must react (for example a designated ingress or upstream node).

Multicast / Point-to-Multipoint: Delivery to a selected group of consumers, for example along a service or forwarding path. Suitable when a defined set of nodes must react together.

Hop-by-hop: Delivery along a series of nodes on a specified path, with each node acting as a relay and possibly a consumer. Suitable for propagating awareness upstream along an affected path.

Scoped Flooding: Dissemination to all nodes within a bounded region of the domain. Suitable for critical events with many interested consumers, with special attention to control overhead and duplicate suppression.

3.5.2. Transport Considerations

Delivery **MAY** reuse existing messaging and transport mechanisms or a new lightweight mechanism **MAY** be defined where existing ones cannot meet the latency or forwarding-plane processing targets. Regardless of the underlying transport, the delivery mechanism is responsible for timely delivery to the intended consumers and for bounding the load it introduces.

Because notifications are most valuable precisely when the network is under stress, the transport **MUST** support prioritization so that notifications are not delayed or dropped behind the very congestion they report. A notification that is queued behind the congested traffic loses most of its value. Prioritization can be realized using existing forwarding-plane mechanisms, including:

- DiffServ marking, for example a dedicated DSCP [[RFC2474](#)] code point mapped to a high-priority or low-latency per-hop behavior (for example Expedited Forwarding [[RFC3246](#)]) along the notification path, so that classification and queuing of notifications can be done in hardware at every hop;
- a strict-priority or low-latency queue, or a dedicated control-class queue, separated from user-data queues so notifications bypass congested data queues;
- at Layer 2, priority marking such as IEEE 802.1p / PCP where the delivery path traverses bridged segments.

The chosen marking and per-hop behavior **MUST** be consistent across the notification domain so that priority is honored E2E within the domain. Operators **MUST** be able to configure the marking, and the markings used for notifications **SHOULD** be reserved so that ordinary traffic cannot claim the same priority and so that notification traffic itself cannot be abused to obtain preferential treatment ([Section 5](#)). Because notifications occupy a high-priority class, their volume **MUST** be bounded by the rate limiting, damping, and filtering of [Section 3.7](#) to avoid starving other control traffic.

Reliability requirements vary by scenario: some events warrant best-effort, low-latency delivery, while others (for example recovery state) may warrant acknowledgement or periodic refresh.

3.5.3. Notification Domain and Isolation

Fast network notifications are confined to a notification domain. The framework requires mechanisms to:

- identify a notification domain and its membership;
- ensure notifications are not propagated outside the domain without explicit policy;
- prevent notifications from one domain being injected into or trusted by another.

Domain scoping bounds the blast radius of both legitimate notification storms and malicious injection, and it aligns the trust boundary with the single administrative control assumed by the charter.

3.6. Discovery, Registration, and Subscription

To deliver notifications only to interested and authorized consumers, the framework supports the following procedures. A deployment **MAY** use configuration, dynamic signaling, or a combination.

Discovery: Originators and consumers determine the existence, identity, and capabilities (event types, encodings, delivery modes) of relevant peers and relays within the domain.

Registration: A node registers as a potential originator or consumer within the domain, establishing the trust and addressing state needed for delivery.

Subscription: A consumer expresses interest in specific event types, locations, paths, flows, or metric thresholds. A subscription-based approach ensures each consumer receives only relevant information, reducing unnecessary overhead. Subscriptions **MAY** be brokered by a controller or established directly between nodes.

These procedures **MAY** be realized by reusing existing protocols where appropriate, or by new mechanisms defined in the protocol specification work.

3.7. Loop Prevention, Filtering, and Damping

Because relays may forward notifications and consumers may relay further, the framework **MUST** provide for:

Loop prevention: Use of origin identity, sequence/epoch indicators, scope limits (for example a hop or region bound), and duplicate suppression so that a notification does not circulate indefinitely.

Filtering and aggregation: Relays **MAY** filter notifications that are not relevant to downstream consumers and **MAY** aggregate multiple related events to reduce volume.

Damping: The framework **MUST** define where responsibility lies for handling rapidly changing conditions, such as a flapping link. Damping **MAY** be applied at the originator, at a transit relay, or required to reach the consumer; the chosen location and its controls **MUST** be specified explicitly. A common policy is to report a degradation immediately but to delay reporting the corresponding recovery for a configurable interval to confirm stability.

4. Realization and Operational Considerations

This chapter describes how the framework relates to existing technologies, the candidate mechanisms that could realize it, the applications it enables, and the scaling and operational considerations that apply when deploying it. It is informational and does not mandate any particular realization.

4.1. Integration with Existing Technologies

A central goal of the framework is integration with existing mechanisms across layers, as required by the charter. Fast network notifications are complementary to these mechanisms.

Layer 2: Link-layer fault and error detection (for example physical-layer alarms, FEC error counters, and interface error statistics) are detection inputs to the originator. Layer 2 protection may act as a consumer's response.

Layer 3 / Routing: Fast network notifications complement IGP/BGP-based dissemination and FRR [RFC4090] [RFC5714]. Whereas a Point of Local Repair acts on a local topology view and may cause congestion on a backup path, a notification can give upstream nodes a wider view before they react. Consumption by routing protocols is a secondary objective and **MUST** preserve routing stability; notifications **MUST NOT** be allowed to induce control-plane churn or instability. Topology and inventory models such as [RFC8345] may provide context for interpreting location and path identifiers.

Layer 4 / Transport: ECN [RFC3168] signals congestion to the transport sender over a full RTT and conveys only a coarse indication. Fast network notifications can deliver richer congestion information to network nodes far sooner than an E2E ECN feedback loop, and may complement transport-layer reactions.

Fast network notifications act inside the network while E2E transport protocols (for example TCP and QUIC congestion control, and ECN/DCTCP-style or RDMA/RoCE congestion management in DC fabrics) act at the endpoints. These two control loops operate concurrently on the same traffic, so the framework **MUST** consider their interaction and ensure FANN-triggered actions do not destabilize or work against transport behavior. Specific concerns include:

- Double reaction: a network reroute or rebalance triggered by a notification can change the path RTT, reorder packets, or shift the bottleneck at the same time the transport sender reacts to loss or ECN marks. Uncoordinated reactions can cause over-correction,

throughput oscillation, or spurious retransmission. FANN-triggered actions **SHOULD** therefore prefer changes that preserve per-flow ordering and avoid abrupt RTT or capacity discontinuities where feasible.

- Masking versus signaling: by mitigating congestion quickly in the network, FANN may suppress the loss or ECN signals that transport congestion control relies on, leaving senders unaware that they are overdriving a path. Therefore, FANN **SHOULD NOT** suppress or rewrite E2E congestion signals (for example clearing ECN marks) unless any negative side effect can be avoided.
- Layering and scope: FANN reacts within a notification domain and on a faster timescale than an E2E loop; it complements but does not replace transport congestion control. The framework treats the transport endpoints as out of scope for action, and any per-flow or per-service identification carried in a notification ([Section 3.4](#)) is used to scope network-side action, not to override endpoint behavior.

The overarching requirement is that FANN remain a net-positive complement to E2E transport: where the interaction cannot be shown to be benign, the conservative choice (for example damping the network reaction, or limiting it to failure rather than congestion events) **SHOULD** be preferred. Detailed coordination between network-side and transport-side reactions is for further study and is expected to be developed with the relevant transport working groups.

Detection and OAM: BFD [[RFC5880](#)] provides fast bidirectional fault detection between endpoints but does not notify other nodes; it can serve as a detection input to the originator. Obtaining faster detection by shortening BFD transmit intervals increases resource consumption, as discussed in [Section 3.3.3](#). IOAM [[RFC9197](#)] and telemetry such as IPFIX [[RFC7011](#)] provide detailed, controller-centric measurement but are not designed for lightweight, rapid alerts to specific nodes for immediate action. Performance metrics may be defined consistently with [[RFC7799](#)]. Fast network notifications fill this gap and feed, rather than replace, telemetry pipelines.

The interaction with each technology, including any required protocol extensions, is expected to be developed in the relevant IETF working groups.

4.2. Candidate Realization Approaches

This section surveys, non-normatively, the classes of mechanism that could realize fast network notifications. It is informational and does not endorse a specific approach; the choice depends on the deployment scenario ([Section 3.2](#)), and a solution **MAY** combine more than one. Concrete protocol work is expected to select and specify mechanisms in separate documents.

Routing-adjacent advertisement decoupled from the IGP: A node advertises link, path, or node state to interested peers using a mechanism that runs alongside, but separately from, the IGP link-state database, so that fast or frequent updates do not perturb routing convergence (see the decoupling principle in [Section 3.1](#)). Work such as [[I-D.zzhang-rtgwg-router-info](#)] illustrates this approach. It suits cases where consumers are routing elements that benefit from a wider view without incurring IGP churn.

IGP/BGP protocol extensions: Existing control-plane protocols could be extended to carry notification information. This reuses deployed machinery but must be weighed carefully against the routing-stability and overhead concerns in [Section 4.1](#), and is generally better suited to slower-changing or control-plane-consumed information than to the fastest forwarding-plane reactions.

In-band / data-plane signaling: Notifications are carried in the forwarding plane, for example in packet headers or lightweight dedicated packets, so that detection, delivery, and consumption can occur in hardware. This offers the lowest latency and best matches the intra-DC scenario, at the cost of requiring forwarding-hardware support and careful scoping. Work such as [\[I-D.clad-rtgwg-ipfrr-aiml\]](#) defines a hardware-accelerated notification protocol operating entirely in the NPU forwarding plane, without CPU intervention, to handle both link failures and congestion within the sub-100-microsecond budget of AI/ML fabrics; [\[I-D.camarillo-rtgwg-lsn\]](#) and [\[I-D.csaszar-rtgwg-ipfrr-fn\]](#) are further examples of fast-notification protocols supporting forwarding-plane protection.

Tunnel- or overlay-based delivery in the WAN: For multi-site or WAN deployments, notifications may be delivered over established tunnels or overlays toward ingress or upstream nodes; work such as [\[I-D.hzh-fantel-wan-tunnel\]](#) explores fast notification in this context for tunnel-based transport.

Telemetry-assisted collection toward the traffic source: Rather than pushing an alert outward from the detecting node, path latency and congestion state are accumulated in-band along the path and returned to the traffic source, so that the source obtains fresh path state and can steer traffic or adjust its congestion response. FALCON [\[I-D.song-rtgwg-falcon\]](#) realizes this by combining in-network telemetry with source routing and collecting the data on the reverse path toward the source, reducing the feedback lag to less than half the baseline RTT; it applies to both DCN and WAN and reuses existing IETF mechanisms.

Each approach trades off latency, hardware dependence, reuse of existing protocols, and impact on routing stability differently, and each fits some scenarios in [Section 3.2](#) better than others. A further open question, noted in the problem statement, is coordination when multiple recipients act on the same notification; the framework treats such action coordination as out of scope and for future study.

4.3. Illustrative Applications

This section sketches, non-normatively, applications that fast network notifications can enable. The actions themselves (protection switching, load balancing, rate adjustment) are out of scope of this framework, as stated in [Section 1.1](#); they are listed here only to illustrate what the notification information in [Section 3.4](#) makes possible and to confirm that the framework's information and delivery models are sufficient to support them.

Upstream (remote) protection: On a notification of a link or node failure, a node several hops upstream of the failure activates a pre-computed backup path, rather than relying solely on local repair at the point of failure. This avoids the traffic hairpinning that purely local alternates (for example LFA or TI-LFA) can introduce and yields more efficient post-failure

paths. Efficient Remote Protection [[I-D.clad-rtgwg-efficient-remote-protection](#)] describes such a mechanism, in which notifications delivered from the point of local repair to remote protecting nodes activate backup paths; it applies both to complete failures and to performance degradations such as reduced link capacity or congestion.

Fast protection in AI/ML fabrics: AI/ML fabrics require convergence within tens of microseconds to avoid the loss and jitter that stall collective communication. Achieving this requires notification propagation and consumption within the forwarding plane, without CPU intervention. [[I-D.clad-rtgwg-ipfrr-aiml](#)] defines a hardware-accelerated notification mechanism of this kind, handling both link failures and congestion.

Graduated load-sharing adjustment: Where a notification carries fine-grained status such as link utilization, available bandwidth, or capacity degradation (rather than a binary up/down indication), an upstream node can adjust the load-sharing weights or ratios across a group of paths in proportion to the reported severity, and can re-adjust as conditions evolve. This turns a coarse failover into a continuous, capacity-aware rebalancing: traffic is shifted away from a degrading or congested path gradually rather than all at once, reducing the risk of overloading the alternate paths. The notification supplies the bandwidth and utilization information; the weight computation and its installation in the forwarding plane are performed by the consuming load-balancing subsystem and are out of scope here. This application reuses the same notification information as remote protection, and the two can be combined, for example load-balancing between a primary and a backup path in proportion to a reported degradation.

These applications share a common requirement on the framework: notifications must be able to convey more than a binary state, must reach the specific upstream or ingress node that performs the action, and must do so faster than the affected traffic's own control loop. The information model ([Section 3.4](#)), recipient and delivery models ([Section 3.5](#)), and scenario discussion ([Section 3.2](#)) are intended to satisfy these requirements.

4.4. Scaling Considerations

The framework must remain effective as the network grows. Scaling pressure arises from network size (the number of nodes and links that may report events), the volume and rate of change of reported information, and the number of consumers. The design assumption is that if anything can go wrong it will, so the system must cope with a high proportion of nodes and links reporting simultaneously.

The framework addresses scale through subscription (delivering only relevant information), scoping and domain isolation (bounding propagation), relay-based filtering and aggregation, damping of rapidly changing conditions, and transport prioritization and rate limiting. Protocol specifications **SHOULD** quantify the load their mechanisms place on the forwarding and control planes under worst-case event conditions.

4.5. Operational Considerations

Fast network notifications introduce additional traffic. During the failures and congestion events they report, the notification system **MUST NOT** exacerbate the situation and **SHOULD** actively assist in mitigating it. Operators **SHOULD** be able to configure which event types trigger notifications, the delivery modes and scopes used, damping and rate-limiting parameters, and prioritization, so that notification behavior aligns with network operation policies.

Management and configuration of the solution are expected to be supported by YANG modules, to be defined as a separate deliverable consistent with the charter. Manageability includes observability of the notification system itself (counts, drops, damping events) so operators can verify it is helping rather than harming.

5. Security Considerations

If not properly authenticated and rate-limited, fast network notifications could be exploited as a denial-of-service vector. An attacker able to inject or flood spurious notifications could trigger unnecessary re-convergence, path changes, or repeated state updates, overwhelming consumers and higher-level applications. An attacker might also induce state flapping to keep an originator busy generating notifications. Notifications may reveal sensitive operational information about the network, either through inspection or by an adversary registering as a consumer.

Accordingly, solutions built on this framework **MUST** provide integrity protection and origin authentication of notifications, and **MUST** apply appropriate rate controls on both sending and receiving. Trust boundaries around domains and subscriptions, authorization of notification sources, and protection of potentially sensitive operational data **MUST** be addressed. Because stronger security can add latency, deployments must consider the trade-off between notification latency and the strength of security on a per-scenario basis. Domain identification and isolation ([Section 3.5](#)) are core to confining notifications to the trusted administrative boundary within which the solution is deployed.

The FANN charter restricts deployment to private networks under a single administrative control. This assumption materially reduces, but does not eliminate, the threat surface, and the framework relies on it deliberately.

Within a single administrative domain the operator controls every originator, relay, and consumer, and the trust boundary coincides with the notification domain ([Section 3.5](#)). This mitigates several concerns directly:

- External injection and spoofing are constrained because all legitimate participants are known to the operator, and the domain boundary can drop notifications arriving from outside it. Admission control at the boundary, combined with the domain isolation already required, addresses the cross-domain injection vector without requiring heavyweight per-message security on every internal hop.

- Information exposure is bounded to the operator's own infrastructure, since notifications are not propagated outside the domain absent explicit policy; the risk of leaking sensitive operational data to third parties is reduced to the cases of misconfiguration or a compromised internal element.
- Trust establishment for discovery, registration, and subscription can lean on existing intra-domain trust infrastructure (for example an operator PKI, shared keys, or fabric-wide identity), rather than inter-domain trust negotiation.

The limited-domain assumption therefore lets the design favor lightweight, low-latency mechanisms internally while concentrating stronger enforcement at the domain boundary. This supports the latency/security trade-off discussed above: the cost of strong per-message protection can be reduced on internal, hardware-forwarded paths where the threat is lower, provided the boundary is properly enforced.

However, the assumption does not remove the need for in-domain protection. Insider threats, compromised or malfunctioning nodes, and the self-inflicted denial of service of a flapping link generating a notification storm all originate inside the trust boundary. The integrity, origin-authentication, rate-limiting, and damping requirements above therefore still apply within the domain; the single-administrative-domain premise should be treated as defense in depth and a means to scope the problem, not as a substitute for those controls.

6. IANA Considerations

This document requires no IANA actions.

7. References

7.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.

7.2. Informative References

- [I-D.camarillo-rtgwg-lsn] Camarillo, P., "Lightspeed Notification Protocol", Work in Progress, Internet-Draft, draft-camarillo-rtgwg-lsn-00, 2026, <<https://datatracker.ietf.org/doc/draft-camarillo-rtgwg-lsn/>>.

- [I-D.clad-rtgwg-efficient-remote-protection]** Clad, F., Filsfils, C., Su, Y., and D. Cai, "Efficient Remote Protection", Work in Progress, Internet-Draft, draft-clad-rtgwg-efficient-remote-protection-00, 2026, <<https://datatracker.ietf.org/doc/draft-clad-rtgwg-efficient-remote-protection/>>.
- [I-D.clad-rtgwg-ipfrr-aiml]** Clad, F. and C. Filsfils, "IP Fast Reroute for AI/ML Fabrics", Work in Progress, Internet-Draft, draft-clad-rtgwg-ipfrr-aiml-00, 2026, <<https://datatracker.ietf.org/doc/draft-clad-rtgwg-ipfrr-aiml/>>.
- [I-D.csaszar-rtgwg-ipfrr-fn]** Csaszar, A., "IP Fast Re-Route with Fast Notification", Work in Progress, Internet-Draft, draft-csaszar-rtgwg-ipfrr-fn-01, 2014, <<https://datatracker.ietf.org/doc/draft-csaszar-rtgwg-ipfrr-fn/>>.
- [I-D.geng-fantel-fantel-gap-analysis]** Geng, X. and J. Dong, "Gap Analysis of Fast Notification for Traffic Engineering and Load Balancing", Work in Progress, Internet-Draft, draft-geng-fantel-fantel-gap-analysis-02, 2026, <<https://datatracker.ietf.org/doc/draft-geng-fantel-fantel-gap-analysis/>>.
- [I-D.geng-fantel-fantel-requirements]** Geng, X. and J. Dong, "Requirements of Fast Notification for Traffic Engineering and Load Balancing", Work in Progress, Internet-Draft, draft-geng-fantel-fantel-requirements-03, 2026, <<https://datatracker.ietf.org/doc/draft-geng-fantel-fantel-requirements/>>.
- [I-D.hzh-fantel-wan-tunnel]** Han, Z., "Fast Notification for Traffic Engineering and Load Balancing for tunnel-based lossless transmission in WAN", Work in Progress, Internet-Draft, draft-hzh-fantel-wan-tunnel-00, 2026, <<https://datatracker.ietf.org/doc/draft-hzh-fantel-wan-tunnel/>>.
- [I-D.ietf-rtgwg-net-notif-ps]** Dong, J., Ed., McBride, M., Ed., and F. Clad, Ed., "Fast Network Notifications Problem Statement", Work in Progress, Internet-Draft, draft-ietf-rtgwg-net-notif-ps-02, 2026, <<https://datatracker.ietf.org/doc/draft-ietf-rtgwg-net-notif-ps/>>.
- [I-D.song-rtgwg-falcon]** Song, H., "Fast Latency and Congestion Notification", Work in Progress, Internet-Draft, draft-song-rtgwg-falcon-00, 2026, <<https://datatracker.ietf.org/doc/draft-song-rtgwg-falcon/>>.
- [I-D.zzhang-rtgwg-router-info]** Zhang, Z., "Advertising Router Information", Work in Progress, Internet-Draft, draft-zzhang-rtgwg-router-info, 2026, <<https://datatracker.ietf.org/doc/draft-zzhang-rtgwg-router-info/>>.
- [RFC2474]** Nichols, K., Blake, S., Baker, F., and D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers", RFC 2474, DOI 10.17487/RFC2474, December 1998, <<https://www.rfc-editor.org/info/rfc2474>>.
- [RFC3168]** Ramakrishnan, K., Floyd, S., and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP", RFC 3168, DOI 10.17487/RFC3168, September 2001, <<https://www.rfc-editor.org/info/rfc3168>>.

- [RFC3246]** Davie, B., Charny, A., Bennett, J.C.R., Benson, K., Le Boudec, J.Y., Courtney, W., Davari, S., Firoiu, V., and D. Stiliadis, "An Expedited Forwarding PHB (Per-Hop Behavior)", RFC 3246, DOI 10.17487/RFC3246, March 2002, <<https://www.rfc-editor.org/info/rfc3246>>.
- [RFC4090]** Pan, P., Ed., Swallow, G., Ed., and A. Atlas, Ed., "Fast Reroute Extensions to RSVP-TE for LSP Tunnels", RFC 4090, DOI 10.17487/RFC4090, May 2005, <<https://www.rfc-editor.org/info/rfc4090>>.
- [RFC5714]** Shand, M. and S. Bryant, "IP Fast Reroute Framework", RFC 5714, DOI 10.17487/RFC5714, January 2010, <<https://www.rfc-editor.org/info/rfc5714>>.
- [RFC5880]** Katz, D. and D. Ward, "Bidirectional Forwarding Detection (BFD)", RFC 5880, DOI 10.17487/RFC5880, June 2010, <<https://www.rfc-editor.org/info/rfc5880>>.
- [RFC7011]** Claise, B., Ed., Trammell, B., Ed., and P. Aitken, "Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information", STD 77, RFC 7011, DOI 10.17487/RFC7011, September 2013, <<https://www.rfc-editor.org/info/rfc7011>>.
- [RFC7799]** Morton, A., "Active and Passive Metrics and Methods (with Hybrid Types In-Between)", RFC 7799, DOI 10.17487/RFC7799, May 2016, <<https://www.rfc-editor.org/info/rfc7799>>.
- [RFC8345]** Clemm, A., Medved, J., Varga, R., Bahadur, N., Ananthakrishnan, H., and X. Liu, "A YANG Data Model for Network Topologies", RFC 8345, DOI 10.17487/RFC8345, March 2018, <<https://www.rfc-editor.org/info/rfc8345>>.
- [RFC9197]** Brockners, F., Ed., Bhandari, S., Ed., and T. Mizrahi, Ed., "Data Fields for In Situ Operations, Administration, and Maintenance (IOAM)", RFC 9197, DOI 10.17487/RFC9197, May 2022, <<https://www.rfc-editor.org/info/rfc9197>>.

Acknowledgements

Author's Address

Haoyu Song
Futurewei Technologies
Email: hsong@futurewei.com